04
Advanced Functional Programming
alias types, module types, contexts, and monads

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Advanced mechanisms of functional programming

Motivation

- FP can be ~~enhanced~~ (potenziata) with mechanisms ~~empowering its applicability~~ (che ne favoriscono l'applicabilità), but still retaining ~~declarativity~~ (dichiaratività)
- many advanced FP mechanisms cross with ideas in OOP
- Scala had the merit of bringing them to mainstream programming
- Question for this lesson: can we build a whole application with pure FP?

Outline

In sintesi, stiamo parlando di come nascondere i dettagli, astrarre il comportamento e gestire il contesto in modo elegante.

Mechanisms/Concepts to empower programming

1. opaque alias types/type members
   - modules internally providing the type of data structure (i.e., abstract data types)
2. module types/inclusive polymorphism (by traits)
   - modules as strategies to pass, with a type and many possible implementations
3. given instances/contextual abstraction
   - modules as contextual parameters, possibly implicitly passed
4. context bounds/ad-hoc polymorphism
   - modules as context bounds, to enable ad-hoc polymorphism via "type classes"
5. for comprehension/monads
   - the "monad" type class to compose functional abstractions in a for-loop

Outline

1. ADTs by alias/opaque types
2. Module types by traits
3. Contextual abstraction by `given`/`using` modifiers
4. Type classes by context bounds
5. `For-yield` and the monad type class
6. A State-monadic little "MVC"

Alias/opaque types and ADTs

come definire strutture dati che siano robuste e protette, ma senza usare necessariamente le classi e l'ereditarietà tipiche di OOP.

Abstract data types

L'approccio "Opaque" dice: "Ti dico che il tipo esiste e ti do gli strumenti per usarlo, ma non ti permetto di vedere com'è fatto dentro".

definisce un tipo di dato specificando

A cornerstone of programming: ~~define a type by~~ name + operations + constructors, abstracting/hiding concrete implementation (in a way that is alternative to encapsulation in OOP)

Idea

oltre ai — la loro implementazione/struttura interna

A module can define types ~~other than~~ methods, and can make ~~their "concretion"~~ opaque to users

Implications

Fuori dal modulo, nessuno sa il vero tipo di un dato (variabile, operazione, etc.)

- we can define datatypes such that their actual implementation is completely hidden
- it is an alternate approach ~~wrt~~ rispetto a explicit algebraic data types we have seen so far (ADT algebrici vs ADT opachi)
- hiding implementation may ~~hamper~~ ostacolare "extendibility" of the set of operations outside the module

Se usi una Algebraic Data Type, puoi scrivere una nuova funzione per quella enum ovunque nel tuo codice usando il match. Se usi un opaque type, non puoi fare pattern matching fuori dal modulo perché non sai cosa c'è dentro. Quindi, se ti serve una nuova operazione fondamentale, devi aggiungerla dentro il modulo originale o tramite extension methods definiti dove il tipo è visibile.

Construct: alias type, with opaqueness

`object O { opaque type T = TImpl ...}`, also with genericity if needed ...

Applications

- any datatype (sequences, streams, optionals) can be defined this way
- could easily define new types reusing transparently existing ones, e.g., sets by algebraic sequences

definire un Set tramite Sequenze (liste)

Euro ADT example

Dentro l'oggetto dove definisci Euro = Int, il compilatore sa che Euro è solo un altro nome per Int.
- Puoi usare tutte le funzioni degli interi perché il modulo deve sapere come manipolare i dati per fornire le operazioni necessarie.
Una volta che esci dal modulo, il compilatore "mette una maschera" al tipo. Per il resto del mondo, Euro è un tipo a sé stante che non ha nulla a che fare con gli interi.

```scala
object EuroADT:
  opaque type Euro = Int // an alias, into this module, opaque outside

  def fromEuroCents(euros: Int, cents: Int): Euro =
    require(euros >= 0 && cents >= 0)
    cents + euros * 100

  def fromDouble(d: Double): Euro =
    require(d >= 0 && d <= 1_000_000)
    (d * 100).toInt // extension method in libs

  extension (e: Euro)
    def asString(): String = "\u0024%d.%2d".format(e / 100, e % 100)
    def sum(e2: Euro): Euro = e + e2

@main def tryEurosADT =
  import EuroADT.*

  val e1: Euro = fromEuroCents(100, 50)
  val e2: Euro = fromDouble(100.0 / 3)
  println(e1.asString()) // E100.50
  println(e2.asString()) // E33.33
  println((e1 sum e2).asString()) // E133.83
  // val e3: Euro = 100 // not allowed because of opaqueness!
  // val e3: Euro = fromEuroCents(-10, 0) // illegal argument exception
```

Costruttori del tipo opaco Euro

Il modulo impone delle regole: non puoi avere Euro negativi. Se provi a crearne uno con dati invalidi, il programma lancia un'eccezione.

Poiché all'esterno Euro non è un Int, non puoi usare '+' o 'println': devi definire esplicitamente cosa è permesso fare

Definisce come visualizzare il tipo.

Definisce l'operazione di somma. All'esterno userai e1 sum e2.

(perché negativo)

Implications

opaque

Using the ADT approach

Nascondere i dettagli permette al designer di cambiare l'implementazione in futuro (es. passare da Int a Long per gestire cifre enormi) senza rompere il codice di chi usa il modulo.

- the designer has to carefully hide all the implementation details of the introduced type
- the client sees a new type
  - it perceives ways of constructing elements of the type
  - it perceives operations to manipulate elements of the type
  - it has no real way to inspect how the type is implemented
- how is `Euro` different from `Int` or `String`?

Il Client non deve preoccuparsi se i centesimi sono gestiti con la virgola o con gli interi; vede solo Euro e le funzioni sum o asString.

Dal punto di vista della CPU, sono identici. Ma dal punto di vista della logica di business, sono mondi opposti

Use it to avoid the "primitive obsession"

La Primitive Obsession è l'abitudine di usare tipi base per concetti complessi ed è un problema perchè viola il Dependency Inversion Principle perché il tuo codice di alto livello (la logica bancaria) finisce per dipendere da dettagli di basso livello (come funzionano gli interi).

- Use of primitives instead of specifically designed types for simple tasks (DIP violation)
- Strings to represent concepts, booleans for 2-valued types, int for "n"-valued types...

Sequence ADT example

```scala
object SequenceADT:
  // implementation of data structure: fully hidden
  private enum SequenceImpl[E]:
    case Cons(head: E, tail: Sequence[E])
    case Nil()
  import SequenceImpl.*

  // the type Sequence is an opaque alias to SequenceImpl
  opaque type Sequence[A] = SequenceImpl[A]

  // constructors
  def cons[E](head: E, tail: Sequence[E]): Sequence[E] = Cons(head, tail)
  def nil[E](): Sequence[E] = Nil[E]()

  // operation as extension methods
  extension [A](seq: Sequence[A])
    def map[B](mapper: A => B): Sequence[B] = seq match
      case Cons(h, t) => Cons(mapper(h), t.map(mapper))
      case Nil()      => Nil()

  // operations as standard methods
  def concat[A](seq1: Sequence[A], seq2: Sequence[A]): Sequence[A] = seq1 match
    case Cons(h, t) => Cons(h, concat(t, seq2))
    case Nil()      => seq2

@main def trySequenceADT =
  import SequenceADT.* // note single, import
  val seq = cons(10, cons(20, cons(30, nil())))
  println(concat(seq, seq.map(_ + 1)))
```

Extension Methods
- Si usano con la "dot notation".
- Servono a definire il comportamento intrinseco del tipo.
- Vantaggio: Migliorano la leggibilità quando si concatenano più operazioni: seq.map(f).filter(g).sort().

del tipo opaco Sequence[A] che all'interno è un alias (sinonimo di SequenceImpl[E]), ma all'esterno è Sequence[A]

(operazioni che si possono eseguire, all'esterno del modulo, su Sequence[A])

del modulo SequenceADT

Metodi standard (Static Method)
- Sono funzioni definite normalmente dentro l'oggetto SequenceADT ma fuori dal blocco extension.
- Si invocano passando tutti gli argomenti tra parentesi. Esempio: concat(seq1, seq2).
- Spesso vengono usati per operazioni che coinvolgono più istanze dello stesso tipo o che fungono da utilità esterne.

Set ADT, backened by algebraic Sequences

```
1  object SetADT:
2    opaque type Set[A] = Sequence[A]
3
4    def fromSequence[A](s: Sequence[A]): Set[A] = s match
5      case Cons(h, t) => Cons(h, fromSequence(t.remove(h)))
6      case Nil()      => Nil()
7
8    def union[A](s1: Set[A], s2: Set[A]): Set[A] = s2 match
9      case Cons(h, t) => Cons(h, union(s1.remove(h), t))
10     case Nil()      => s1
11
12   def intersection[A](s1: Set[A], s2: Set[A]): Set[A] = s1 match
13     case Cons(h, t) if s2.contains(h) => Cons(h, intersection(t, s2.remove(h)))
14     case Cons(_, t) => intersection(t, s2)
15     case Nil()      => Nil()
16
17   extension [A](s: Set[A])
18     def contains(a: A): Boolean = s match
19       case Cons(h, t) if h == a => true
20       case Cons(_, t)           => t.contains(a)
21       case Nil()                => false
22     def remove(a: A): Set[A] = s.filter(_ != a)
23     def toSequence(): Sequence[A] = s
```

A livello di memoria e prestazioni, un Set è una Sequence. Tuttavia, marcandolo come opaque, impediamo all'utente di trattarlo come una lista ordinata o di aggiungervi duplicati liberamente. L'utente può interagire con il Set solo tramite le regole che definiamo noi.

fromSequence è un costruttore.

Trasforma una sequenza sporca (magari con duplicati) in un Set pulito, prendendo la testa (h), e la aggiunge al risultato solo dopo averla rimossa dal resto della coda (t.remove(h)). È un processo di de-duplicazione ricorsivo.

Standard Methods

Per unire due set, non fa una semplice concatenazione (che creerebbe duplicati), ma fonde gli elementi assicurandosi che ogni elemento sia unico.

Pattern matching con una clausola if: un elemento fa parte dell'intersezione solo se è presente in entrambi i set.

Trasforma il Set in una sequenza normale per permettere, ad esempio, di stamparne i valori o iterarli.

Usage of sets

```scala
@main def trySetADT =
  import SetADT.*
  val s1: Set[Int] = fromSequence(Cons(10, Cons(20, Cons(10, Cons(30, Nil())))))
  val s2: Set[Int] = fromSequence(Cons(10, Cons(11, Nil())))
  // val s3: Set[Int] = Cons(10, Nil()) // because Set is defined opaque
  println(s1.toSequence()) // (10, 20, 30)
  println(s2.toSequence()) // (10, 11)
  println(union(s1, s2).toSequence()) // (10, 20, 30, 11)
  println(intersection(s1, s2).toSequence()) // (10)
```

On ADTs vs algebraic data types

Quando è meglio nascondere tutto (ADT) e quando è meglio mostrare tutto (Algebraic Data Type)?

ADT

opaque type

- are a more standard software engineering approach
- promote abstraction and modularity

Se nascondo come è fatto un Set, posso cambiare la sua implementazione (da lista ad albero) senza che nessuno debba cambiare una riga di codice fuori dal modulo.

- treat operations and constructors uniformly

perché sono tutti semplici funzioni del modulo.

- from there, one more naturally evolves to type classes

Poiché l'ADT è un'astrazione pura, si presta bene a essere usato con le Type Classes, dove definisci comportamenti per quel tipo.

Exposed algebraic data types

case class o enum

- somewhat more reflect Scala approach in libraries

Tutti vedono che una List è composta da Cons e Nil.

- more handy to flexibly add algorithms outside the definition

Se voglio scrivere una funzione con argomento la lista, posso farlo in un file separato usando il pattern matching. Con un ADT opaco, non potrei farlo perché non saprei "smontare" la lista.

- from there, one more naturally evolves to the OOP approach

è molto facile aggiungere sottoclassi o metodi direttamente nel tipo, scivolando verso lo stile ad oggetti.

In the following

- for sequences/streams we assume the algebraic data types version
- the rest of this lesson keeps discussing from ADTs

Outline

1. ADTs by alias/opaque types
2. Module types by traits
3. Contextual abstraction by `given`/`using` modifiers
4. Type classes by context bounds
5. `For-yield` and the monad type class
6. A State-monadic little "MVC"

Module types

Se gli ADT (visti prima) servono a nascondere come è fatto un dato, i Module Types servono a definire cosa un modulo deve saper fare, indipendentemente da chi lo implementa: separa la specifica di un componente dalle sue varie implementazioni.

Idea

Separare il "Cosa" dal "Come"

Modules, might be given a type, separating concept from implementation, and such that many interchangeable implementations may exist

Un Module Type è un contratto (in Scala si usa un trait). Definisce le signature dei metodi ma non il loro corpo.

Implications

- we can provide many implementations of a given module, trading off various aspects
- this allows to pass module implementations to functions to make them more abstract (Pattern Strategy)
- an application may decide which implementation to use each time in base al contesto.
- very useful in ADTs
- allows us to deal with OCP and DIP in a pure FP setting

Immagina di avere il nostro SetADT di prima. Se lo trasformi in un Module Type, puoi avere:
- Un'implementazione FastSet (che usa gli Hash ed è velocissima).
- Un'implementazione SimpleSet (che usa le sequenze ed è facile da testare).

Le funzioni che ricevono un Set non sapranno (e non vorranno sapere) quale delle due stanno usando.

Constructs

```
trait ModuleType { < method signatures > }
object ModuleImpl extends ModuleType { ... }
```

Concetto (contratto) Implementazione rispettando il contratto

...

```
def meth(o: ModuleType)(...) = { ... o.m(...) ... }
meth(ModuleImpl)(...)
```

molto simile al Pattern Strategy

Module types at work: LoggerModuleType

Questo esempio è la traduzione pratica del Pattern Strategy applicato ai moduli.

```scala
object LoggerModules:

  // type for modules exposing definition of a log method (no implementation)
  trait LoggerModuleType:
    def log(s: String): Unit

  // implementation of a module adhering to LoggerModuleType
1 object BasicLoggerModule extends LoggerModuleType:
    def log(s: String): Unit = println(s)

  // implementation of a module adhering to LoggerModuleType
2 object EmptyLoggerModule extends LoggerModuleType:
    def log(s: String): Unit = {}

  // method call requiring indication of a LoggerModuleType (Strategy Pattern)
  def loggedCall[A, B](name: String, f: A => B, a: A)(LoggerModule: LoggerModuleType): B =
    LoggerModule.log(s"calling function $name with input $a")
    f(a)

@main def tryLoggers =
  import LoggerModules.*

  // passing the desired module
  println(loggedCall("sin", Math.sin(_), 0.5)(BasicLoggerModule))
  println(loggedCall("sin", Math.sin(_), 0.5)(EmptyLoggerModule))
```

Questo è il contratto. Stai dicendo: "Non mi importa chi sei o come funzioni, ma se vuoi essere un Logger, devi avere un metodo log che accetta una String e non restituisce nulla". È pura astrazione. Non c'è codice eseguibile qui, solo la definizione di un'abilità.

due strategie opposte per soddisfare il contratto. Entrambi sono oggetti (object), quindi sono istanze singole (Singleton) che implementano il tipo del modulo.

Module types for algorithms: MathModule and a basic implementation

```scala
object MathModules :

  // type of math modules
  trait MathModuleType:
    def factorial(n: Int): Int
    def exp(base: Double, power: Int): Double

  // non-tail recursive solutions without input checks
  object BasicMathModule extends MathModuleType:

    def factorial(n: Int): Int =
      if n == 0 then 1 else n * factorial(n - 1) // note if-then-else

    def exp(base: Double, power: Int): Double =
      if power == 0 then 1 else base * exp(base, power - 1)
```

Module types for algorithms: an alternate implementation

```scala
// tail recursive solutions with input checks
object ProductionMathModule extends MathModuleType:

  def factorial(n: Int): Int =
    @scala.annotation.tailrec
    def _fact(n: Int, temp: Int): Int =
      if n == 0 then temp else _fact(n - 1, temp * n)
    n match
      case _ if n >= 0 => _fact(n, 1)

  def exp(base: Double, power: Int): Double =
    @scala.annotation.tailrec
    def _exp(p: Int, temp: Double): Double =
      if p == 0 then temp else _exp(p - 1, temp * base)
    power match
      case _ if base >= 0 => _exp(power, 1)
```

Module types for algorithms: use of MathModule

```scala
@main def tryMathModule =
  import MathModules.*

  // probability of having x successes over n trials, where each success has prob. p
  // abstracting from specific implementation of the math module
  def binomialProbability(mm: MathModuleType)(n: Int, x: Int, p: Double): Double =
    mm.factorial(n) / mm.factorial(x) / mm.factorial(n-x) *
    mm.exp(p, x) * mm.exp(1-p, n-x)

  // probability of 6 heads on 10 coin tosses
  println:
    binomialProbability(BasicMathModule)(10, 6, 0.5)
  println:
    binomialProbability(ProductionMathModule)(10, 6, 0.5)
```

SetADT abstraction and use

```
1  trait SetADT:
2    type Set[A]
3    def fromSequence[A](s: Sequence[A]): Set[A]
4    def union[A](s1: Set[A], s2: Set[A]): Set[A]
5    def intersection[A](s1: Set[A], s2: Set[A]): Set[A]
6    extension [A](s: Set[A])
7      def contains(a: A): Boolean
8      def remove(a: A): Set[A]
9      def toSequence(): Sequence[A]
```

Stai dichiarando che ogni modulo che implementa SetADT deve definire cosa sia un Set, ma il trait generico non lo sa (chi usa il trait SetADT vede solo il nome Set, non la struttura sottostante.).

```
1  @main def trySetADTModule =
2    import Sets.*
3    val setADT: SetADT = BasicSetADT
4    import setADT.*
5
6    val s1: Set[Int] = fromSequence(Cons(10, Cons(20, Cons(10, Cons(30, Nil())))))
7    val s2: Set[Int] = fromSequence(Cons(10, Cons(11, Nil())))
8    // val s3: Set[Int] = Cons(10, Nil()) // because Set is defined opaque
9    println(s1.toSequence()) // (10, 20, 30)
10   println(s2.toSequence()) // (10, 11)
11   println(union(s1, s2).toSequence()) // (10, 20, 30, 11)
12   println(intersection(s1, s2).toSequence()) // (10)
```

Scegliamo l'implementazione

Esportiamo i metodi di QUELLA specifica istanza

il compilatore ci impedisce di assegnare una Sequence direttamente a un Set.

SetADT (usual) implemetation

```
1  object BasicSetADT extends SetADT:
2    opaque type Set[A] = Sequence[A]
3
4    def fromSequence[A](s: Sequence[A]): Set[A] = s match
5      case Cons(h, t) => Cons(h, fromSequence(t.remove(h)))
6      case Nil() => Nil()
7
8    def union[A](s1: Set[A], s2: Set[A]): Set[A] = s2 match
9      case Cons(h, t) => Cons(h, union(s1.remove(h), t))
10     case Nil() => s1
11
12   def intersection[A](s1: Set[A], s2: Set[A]): Set[A] = s1 match
13     case Cons(h, t) if s2.contains(h) => Cons(h, intersection(t, s2.remove(h)))
14     case Cons(_, t) => intersection(t, s2)
15     case Nil() => Nil()
16
17   extension [A](s: Set[A])
18     def remove(a: A): Set[A] = s.filter(_ != a)
19     def contains(a: A): Boolean = s match
20       case Cons(h, t) if h == a => true
21       case Cons(_, t) => t.contains(a)
22       case Nil() => false
23     def toSequence(): Sequence[A] = s
24
```

Definisce l'interfaccia (il "Cosa").

Fornisce l'implementazione specifica (il "Come").

Questa informazione rimane "segreta" dentro BasicSetADT.

Costruttore: prendo dei dati e creo il Set[A] (tipo opaco)

Poiché siamo dentro il modulo, il compilatore ci permette di fare il match sul tipo opaco Set come se fosse una Sequence.

Outline

1. ADTs by alias/opaque types
2. Module types by traits
3. Contextual abstraction by `given`/`using` modifiers
4. Type classes by context bounds
5. `For-yield` and the monad type class
6. A State-monadic little "MVC"

Given/using and contextual abstraction

L'idea di base: alcuni parametri di una funzione non sono " dati" su cui operare, ma "contesto" in cui operare.

Idea

Immagina una funzione che deve salvare un utente nel database. Dato: L'utente da salvare (cambia ogni volta). Contesto: La connessione al database o il Logger (sono quasi sempre gli stessi per tutta l'esecuzione). Invece di passare il Logger a mano in ogni singola funzione (creando codice ripetitivo), lo marchiamo come contesto. Il compilatore si occuperà di "riempire il buco" per noi.

Contextual abstraction is the idea that certain inputs of a function are actually just "context", ~~hence~~ quindi one might want to explicitly tag that inputs (in definition and call), and also wants implicit ways of defining/passing them

Implications

Grazie a

- ~~by~~ contextual abstraction, ~~one more easily captures~~ è più facile acquisire additional arguments/strategies to methods
- it is a base for a key mechanism known as type classes, we will see later

Perché le Type Classes permettono di definire un comportamento e di passarlo implicitamente a funzioni.

Constructs

```
def meth(arg: Type)(using ctx: CtxType){ ... }, definition
meth(actualArg)(using actualCtx), call with explicit context
...
given givenName: CtxType = actualCtx, definition of a canonical term
meth(actualArg), call with implicit context
```

un 'canonical term' è un oggetto definito con la parola chiave given

Details

In Scala, una funzione può avere più gruppi di parentesi (Currying). La regola è che il gruppo using deve essere l'ultimo.

- `using` clause is relative to all (curried) parameter, typically at the end
- definition of a canonical term can actually occur with alternate syntax (e.g., avoiding `givenName`)

Perché alla fine? Perché prima fornisci i "dati" e solo alla fine lasci che il compilatore aggiunga il "contesto".

Sintassi Anonima (Senza nome) is usa per essere passata a una clausola using. Se non devi mai chiamare dot method manualmente nel tuo codice, non serve dargli un nome.

Contextual parameters (not very common...)

```scala
object ContextualParameters:

  /* A standard max over a sequence, with an additional "configuration"
   * or "contextual" parameter, specified by using clause. Note the using
   * clause is not necessary in the recusive call, since the context can be
   * considered the same.
   */
  def max(seq: Sequence[Int])(using orElse: Int): Int = seq match
    case Cons(h1, Cons(h2, t)) => max(Cons(if h1 > h2 then h1 else h2, t))
    case Cons(h1, Nil())       => h1
    case _                     => orElse

  given standardDefault: Int = -1 // if using an Int is needed, suggest to take this!

@main def tryContextualParameters() =
  import ContextualParameters.*

  // input and context are passed with specific syntax
  println:
    max(Cons(10, Cons(30, Cons(20, Nil()))))(using -1)
  println:
    max(Nil())(using -1)

  import ContextualParameters.given // or ContextualParameters.standardDefault

  println:
    max(Cons(10, Cons(30, Cons(20, Nil())))) // no need for the using clause
```

Poiché orElse è un parametro using, il compilatore vede che manca. Invece di dare errore, guarda se nel contesto locale c'è un intero disponibile. Non devi passare "a mano" la configurazione in ogni passo della ricorsione.

"Usa -1 come valore di default se la lista è vuota". È il modo classico per forzare un valore.

Un normale import ContextualParameters.* non importa i given. Devi importarli esplicitamente.

Dopo l'import, non serve più il using. Il compilatore trova standardDefault e lo inserisce lui.

Contextual strategies

Invece di decidere internamente che max deve restituire il numero più grande, deleghiamo la decisione a una funzione ordering. La funzione max diventa un contenitore vuoto: sa come scorrere la lista, ma non sa come confrontare due elementi.

```
1  object ContextualStrategies:
2
3    /* a standard max over a sequence of Int, passing a strategy for ordering
4     * seq as a "context".
5     * max could be made generic, not developed here...
6     */
7    def max(seq: Sequence[Int])(using ordering: (Int, Int) => Boolean): Int = seq match
8      case Cons(h1, Cons(h2, t)) => max(Cons(if ordering(h1, h2) then h1 else h2, t))
9      case Cons(h1, Nil()) => h1
10
11   @main def tryContextualStrategies() =
12     // context is passed with specific syntax
13     println:
14       max(Cons(10, Cons(30, Cons(20, Nil()))))(using _ > _)
15     println:
16       max(Cons(10, Cons(30, Cons(20, Nil()))))(using _ < _)
17
18     // the given defined here needs not be imported
19     // name of given is not very helpful, it can be avoided
20     given ((Int, Int) => Boolean) = _ > _
21
22     println:
23       max(Cons(10, Cons(30, Cons(20, Nil()))))
```

h1 maggiore di h2?

h1 minore di h2?

Sintassi Anonima: Non abbiamo dato un nome al given. Abbiamo solo detto: "Se serve una funzione che confronta due interi, usa questa". Poiché è definito nello stesso scope in cui viene usato, non serve importarlo.
Risoluzione automatica: Alla riga 23, chiamiamo max(...) senza il secondo gruppo di parentesi. Il compilatore vede che manca una funzione (Int, Int) => Boolean e "pesca" automaticamente quella definita alla riga 20.

Contextual modules

```scala
object ContextualModules:

  trait IntOrderingModuleType:
    def greater(t1: Int, t2: Int): Boolean

  def max(seq: Sequence[Int])(using ordering: IntOrderingModuleType): Int = seq match
    case Cons(h1, Cons(h2, t)) => max(Cons(if ordering.greater(h1, h2) then h1 else h2, t))
    case Cons(h1, Nil()) => h1

  // defining a module implementing the contract
  object MyIntOrderingModule extends IntOrderingModuleType:
    def greater(t1: Int, t2: Int): Boolean = t1 > t2

  // defining a canonical term taking an existing module
  given iom: IntOrderingModuleType = MyIntOrderingModule

@main def tryContextualModules =
  import ContextualModules.*
  // importing all given
  import ContextualModules.given
  // importing a specific given by: import ContextualModules.iom
  // importing just * would not work!

  println:
    max(Cons(10, Cons(30, Cons(20, Nil()))))

  val ord = summon[IntOrderingModuleType] // gives ContextualModules.iom
```

La funzione summon fa questo "Trova e restituiscimi l'istanza given di questo tipo che è attualmente attiva".

Contexts as injected dependencies

Dependency Injection

```
1
2  @main def tryContextualMathModule =
3    import u04.MathModules.*
4
5    // probability of having x successes over n trials, where each success has prob. p
6    // abstracting from specific implementation of the math module
7    def binomialProbability(n: Int, x: Int, p: Double)(using mm: MathModuleType): Double =
8      mm.factorial(n) / mm.factorial(x) / mm.factorial(n-x) *
9      mm.exp(p, x) * mm.exp(1-p, n-x)
10
11   // injecting the dependency by a given!!
12   given MathModuleType = ProductionMathModule
13
14   // probability of 6 heads on 10 coin tosses
15   println:
16     binomialProbability(10, 6, 0.5)
17   println:
18     binomialProbability(10, 6, 0.5)
```

dependency

Contextual Injection

Outline

1. ADTs by alias/opaque types
2. Module types by traits
3. Contextual abstraction by `given`/`using` modifiers
4. Type classes by context bounds
5. `For-yield` and the monad type class
6. A State-monadic little "MVC"

Context bounds and type classes

il Polimorfismo Ad-hoc tramite le Type Classes.

Sta dicendo che una funzione può accettare un tipo T solo se esiste un'implementazione di quel modulo per quel tipo T disponibile nel codice.

Idea

(example: trait GenericModuleType[T] ...)

A module type can be generic. When this is used as a context, ~~it resembles a~~ "context bound", ~~namely a~~ constraint on the type variable, ~~stating that an instantiation is admissible~~ only if the given (generic) context is available in scope.

assume la forma di un

ovvero un

secondo cui un'istanziazione è ammissibile

Implications

- essentially a `using` clause taking a `GenericModuleType[T]` parameter (simile ad usare using nei parametri e solo [T] nel nome della funzione)
- ~~such a context~~ `GenericModuleType[_]` can be interpreted as an "a posteriori" extension of type T, namely, a so-called "type class" (tale contesto)
- it is a programmable way of supporting so-called ad-hoc polymorphism (È un modo programmabile per far sì che una funzione si comporti in modo diverso a seconda del tipo T che riceve (ad esempio: una funzione somma che sa come sommare numeri, ma anche come unire stringhe).
- ultimately, it calls for defining "concise data types" extensible later, where/if needed

Constructs

`trait GenericModuleType[T]{ ... }`, type class definition

`def meth[T: GenericModuleType](arg: Type){ ... }`, method with ad-hoc polymorphism

"Accetto ogni T, a patto che ci sia un given di tipo GenericModuleType per lui".

Context Bound

...

`summon[GenericModuleType[T]]`, ~~recovering~~ recupero. the "witness" implementation of the extension of T if needed

È il comando per "pescare" il modulo e usarne i metodi, dato che col Context Bound non gli abbiamo dato un nome esplicito

The Ordered type class

L'estensione a posteriori è il meccanismo (aggiungere capacità dopo che il tipo è stato definito tremite given), mentre il polimorfismo ad-hoc è il risultato (la funzione si adatta a tipi diversi che non hanno una radice comune).

```scala
// Ordered as a generic module type, namely, a so called "type class"
trait Ordered[T]:
  def greater(t1: T, t2: T): Boolean

// using-oriented style of definition
def maxWithUsing[T](seq: Sequence[T])(using ordered: Ordered[T]): T = max(seq)

// equivalent definition with context bound, recalling "ad-hoc polymorhism"
// it can be called on T only if an Ordered[T] is available in scope
def max[T: Ordered](seq: Sequence[T]): T = seq match
  case Cons(h1, Cons(h2, t)) => max(Cons(if summon[Ordered[T]].greater(h1, h2) then h1 else h2, t))
  case Cons(h1, Nil()) => h1

// defining an Ordered "extension" for type Int
given Ordered[Int] with
  def greater(t1: Int, t2: Int): Boolean = t1 > t2

// doing the same for String, in a different style
object MyOrderedString extends Ordered[String]:
  def greater(t1: String, t2: String): Boolean = t1 > t2
given Ordered[String] = MyOrderedString

@main def tryContextBound =
  // the necessary given are already in scope
  println:
    max(Cons(10, Cons(30, Cons(20, Nil()))))
  println:
    max(Cons("a", Cons("c", Cons("d", Nil()))))
  // println(max(Cons(1.0, Cons(3.0, Cons(5.0, Nil())))) // not working
```

Polimorfismo Ereditario (Sottotipazione): Una funzione accetta Animale e funziona per Cane e Gatto perché sono animali. Il comportamento è scritto "dentro" di loro.

Polimorfismo Ad-Hoc: Una funzione accetta un tipo T generico, ma il comportamento cambia appositamente in base a quale T passi, grazie a un modulo esterno (la Type Class).

Due estensioni a posteriori di Ordered per Int e per String

Non dà errori perchè esiste nello scope esterno un Ordered[Int]

Non dà errori perchè esiste nello scope esterno un Ordered[String]

Dà errore perchè non esiste nello scope esterno un Ordered[Double]

The Showable type class

Cosa succede quando scrivi 10.show()? Senza questa riga di estensione, saresti costretto a scrivere ogni volta il codice lungo e noioso: summon[Showable[Int]].show(10)
Grazie a questa estensione, il compilatore fa il lavoro sporco per te:
- Vede 10.show().
- Capisce che 10 è un Int.
- Cerca un'estensione per Int che si chiami show.
- Trova la tua extension [A: Showable].
- Verifica se esiste un given Showable[Int].
- Riscrive internamente il tuo codice 10.show() in Showable[Int].show(10).

```scala
object Showables:
  // the Showable type class
  trait Showable[T]:
    def show(t: T): String

  // algorithms/operations on showables
  object Showable:

    extension [A: Showable](a: A)
      def show(): String = summon[Showable[A]].show(a)

    def showPair[A: Showable, B: Showable](t: (A, B)): String = t match
      case (a, b) => "(" + a.show() + ", " + b.show() + ")"

    def showSequence[A: Showable](seq: Sequence[A]): String = seq match
      case Sequence.Cons(h, t) => "|" + h + showSequence(t)
      case Sequence.Nil() => ":"
```

Posso usare a.show() se a è di tipo A e nello scope esiste un given Showable[A]

Standard Methods con Polimorfismo ad-hoc

Il Polimorfismo Ad-hoc è un concetto di programmazione in cui una funzione può assumere comportamenti diversi per tipi diversi, ma non perché questi tipi hanno un antenato comune (come l'ereditarietà).

Si chiama "Ad-hoc" (che in latino significa "per questo scopo specifico") perché il comportamento della funzione viene definito appositamente per ogni tipo che vuoi supportare.

The Showable type class: given instances and use

```scala
object ShowableGivenInstances:
  import Showables.*, Showable.*

  // canonical terms for Int, String, Student
  given Showable[Int] with
    def show(i: Int): String = "" + i

  given Showable[String] with
    def show(s: String): String = s

  case class Student(name: String, id: Int)

  given Showable[Student] with
    def show(s: Student): String = s match
      case Student(n, i) => "stud("+ n.show() + ", " + i.show()+")"

@main def tryShowable =
  import Showables.*, Showable.*
  import ShowableGivenInstances.{*, given}

  // note it seems like we have actually dynamically extended Int, String...
  println(10.show())
  println("hello!".show())
  println(Student("mario", 201).show())
```

Devo importare anche Showable, perchè in Scala 3, i metodi definiti dentro una extension sono visibili solo se l'estensione stessa è nello scope.

Higher kinded types: "generalising genericity"

generalizzare la genericità

La Gerarchia dei "Kind" (Tipi di Tipi)

```scala
// 0-kinded types, that is, standard types
trait FactorialModuleType:
  def factorial(n: Int): Int

object BasicFactorialModule extends FactorialModuleType:
  def factorial(n: Int): Int = if n == 0 then 1 else n * factorial(n-1)

val f = BasicFactorialModule.factorial(5)

// 1-kinded types, that is, types generic over a 0-kinded type
trait Showable[T]:
  def show(t: T): String

object ShowableInt extends Showable[Int]:
  def show(i: Int): String = "" + i

val s = ShowableInt.show(5)

// 2-kinded types, that is, types generic over a 1-kinded type
trait Filterable[T[_]]:
  def filter[A](t: T[A])(f: A => Boolean): T[A]

import u03.extensionmethods.Optionals.*, Optional.*
object FilterableOptional extends Filterable[Optional]:
  def filter[A](t: Optional[A])(f: A => Boolean): Optional[A] = t match
    case Just(a) if f(a) => Just(a)
    case _ => None()

val opt = FilterableOptional.filter(Just(11))(_ > 10)
```

I tipi standard

I tipi generici classici

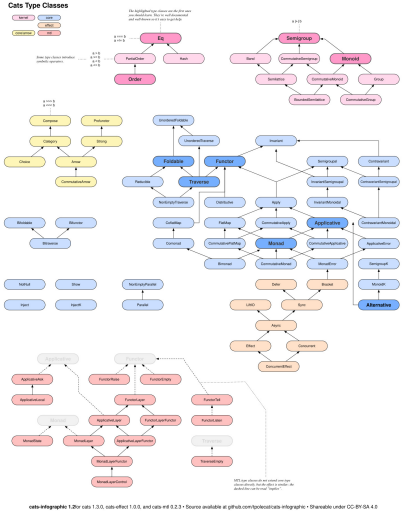
mi interessa definire il tipo della "scatola", non quello che c'è dentro, che sicuramente sarà stato definito da qualche altra parte

Higher-Kinded Types

In pratica, stai astraendo sopra l'astrazione. Invece di preoccuparti di cosa c'è dentro la scatola (Int, String), ti preoccupi di com'è fatta la scatola (List, Optional, Future) e di come puoi manipolarla.

Type classes in the Cats library

Cats è una libreria Scala che fornisce astrazioni di programmazione funzionale tramite type classes per il polimorfismo ad hoc.



cats-infographic 1.2for cats 1.3.0, cats-effect 1.0.0, and cats-mtl 0.2.3 • Source available at github.com/tpolecat/cats-infographic • Shareable under CC-BY-SA 4.0

Outline

1. ADTs by alias/opaque types
2. Module types by traits
3. Contextual abstraction by `given`/`using` modifiers
4. Type classes by context bounds
5. `For-yield` and the monad type class
6. A State-monadic little "MVC"

Monads

una Monade è semplicemente un pattern di design per gestire sequenze di operazioni su dati "impacchettati" (come Option, List, Future). Consente di strutturare i calcoli in sequenza, incapsulando effetti collaterali, gestione degli errori o stati all'interno di un contenitore

La for-comprehension in Scala è uno "zucchero sintattico", ovvero un modo più leggibile per scrivere una catena di operazioni su Monadi. Anche se sembra un ciclo for imperativo (come in Java o C), in realtà è un costrutto funzionale che serve a estrarre valori da dei contenitori (Monadi), lavorarci sopra e impacchettare il risultato in un nuovo contenitore.

Everybody talks about monads (sources intentionally neglected)

- .."it is the one-for-all recipe"
- .."it is the ~~single~~ unico design pattern for FP"
- .."it is a way of programming semicolon in for comprehension"
- .."it is variable binding mechanism"
- .."it can bring logic features into FP"
- .."it is a tool for packing imperative aspects as pure FP"

In Scala, la Monade permette di concatenare istruzioni (tramite flatMap) dentro un for, gestendo automaticamente cosa succede se qualcosa fallisce (come un None in un Option).

Cose "sporche" come l'Input/Output (leggere un file, scrivere a video) o gli stati variabili possono essere trattate come valori puri se chiuse dentro una Monade (come IO o State).

History

- In category theory: an *endofunctor* with two natural transformations $I \leftarrow T$; $T \circ T \leftarrow T$, and certain laws (monadic laws, presented later)
- Eugenio Moggi (UNIGE) described in 1991 their use in programming
- Haskell 3.1 introduces monads to handle I/O with lazy evaluation
- Monads are shown to be fruitfully used in FP in a variety of cases
  - sequences, optionals, streams, stateful evaluation, futures, I/O, parsers
- Scala collections are monadic, and this is used in for comprehension
- Libraries like `scalaz` and `cats` provides full support to monads

For comprehension, and `flatMap/map`

```scala
def getRandom(): Option[Double] =
  if Math.random() < 0.9 then Some(Math.random) else None

// preview of for comprehension with scala Option[E]
val sum: Option[Double] =
  for
    x <- getRandom()
    y <- getRandom()
    z <- getRandom()
  yield x+y+z
println(sum) // Some(x+y+z) or None(), probabilistically

// preview of for comprehension with scala List[E]
val list: List[String] =
  for
    x <- List(10,20,30)
    y <- List("A","B","C")
  yield x+y
println(list) // List(10A, 10B, 10C, 20A, 20B, 20C, 30A, 30B, 30C)

// equivalent to
val list2: List[String] =
  List(10,20,30).flatMap(x =>
    List("A","B","C").map(y =>
      x+y))
println(list2) // List(10A, 10B, 10C, 20A, 20B, 20C, 30A, 30B, 30C)
```

Se getRandom() restituisce None anche solo una volta, l'intera operazione si interrompe immediatamente.

Se tutti e tre i valori vengono generati con successo, avrai Some(x+y+z), se no None.

In ogni Monade, la flatMap viene eseguita tra una riga ed un'altra (come se implementassimo il ";" tra un istruzione ed un altra in Java): getRandom().flatMap(x => getRandom().flatMap(y => getRandom().map(z => map(x + y + z))))

What monads can do for you, generally?

L'operatore unit (in Scala spesso chiamato apply o pure) è il punto di ingresso. Prende un valore "nudo" e lo mette nel contesto monadico.
- Esempio: Prendi il numero 5 e lo metti in un Option: Option(5).
- Perché serve? Perché per far entrare un dato in una catena di montaggio monadica, deve prima essere "impacchettato".

A monad

una Monade è un "gestore di contesto". Immagina di avere un valore (un numero, una stringa) dentro una scatola. La Monade è il regolamento che spiega come passare quel valore da una scatola all'altra senza mai doverle rompere o gestire manualmente gli errori.

- a computational "package" for elements of a type `A`, giving:
  - an operator `unit` that creates the "package" around a value of type `A`
  - an operator `bind` (also called `flatMap`) that takes. . .
    - a "package" (monad) of `A`
    - a function from `A` to another "package" of `B`
  - .. and performs as follows:
    - takes values out of the input monad
    - apply to them the function, ultimately returning one "package" of `B`
- it is a functional-oriented and programmable composition mechanism for such packages

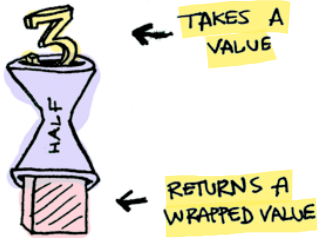
Il suo lavoro è:
- Estrarre: Guarda dentro la scatola A.
- Applicare: Se c'è qualcosa, passa il valore a una funzione che crea una scatola B.
- Appiattire: Invece di darti una "scatola dentro una scatola", ti restituisce direttamente la scatola B.

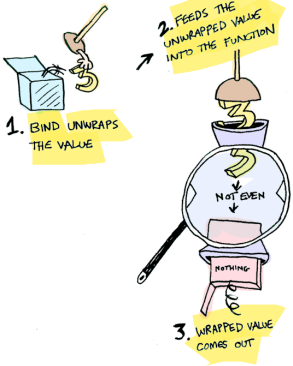
General meaning of `flatMap`

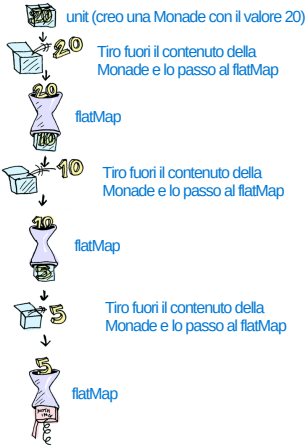
Prima esegue Map e poi Flatten
- Map: Applica la logica alla riga successiva.
- Flatten: Impedisce che il risultato diventi una scatola dentro una scatola (es. List(List(1))).

- "`for x <- m; ...`" becomes "`m.flatMap(x => ...)`"
- in a sense, by `flatMap` we implement "our" meaning of ";" in for comprehensions involving `m`'s type
- essentially, we program what to extract from the monad and how to pass it to next stages

Monads by pictures (http://adit.io)







Monads and monadic laws

```scala
// F[_] is called a "type constructor", or 1-kinded type
// Monad is generic in a type constructor named F
trait Monad[F[_]]:
  // two constructs for monads, namely, its true "definition"
  def unit[A](a: => A): F[A]
  def flatMap[A, B](ma: F[A])(f: A => F[B]): F[B]

// monadic laws that must hold:
  // Left identity: flatMap(unit(a),f) === f(a)
  // Right identity: flatMap(m,unit(_)) === m
  // Associativity: flatMap(flatMap(m,f),g) ===
  //                flatMap(m,x=>flatMap(f(x),g))

// map as a key derived operation
  def map[A, B](ma: F[A])(f: A => B): F[B] =
    flatMap(ma)(a => unit(f(a)))
```

una Monade non è un tipo singolo (come Int), ma un contenitore (come List, Option o Future).

Per essere una Monade, quel contenitore deve saper fare due cose: impacchettare (unit) e concatenare (flatMap).

Prende un valore di tipo A e lo mette nella scatola.

Prende una scatola, ne estrae il contenuto di tipo A, ci applica una funzione che crea una nuova scatola, e restituisce quest'ultima (senza annidamenti, cioè non una scatola con dentro una scatola).

Queste leggi servono a garantire che la Monade si comporti in modo coerente.

La scatola non deve alterare il dato (unit(x).flatMap(f) == f(x)).

Devo riottenere la stessa scatola, se prendo il contenuto della scatola e lo rimetto nella scatola(m.flatMap(unit) == m)

Non importa l'ordine con cui raggruppi le operazioni di concatenazione; il risultato finale deve essere lo stesso. È quello che permette al for di funzionare bene su molte righe ((m.flatMap(f)).flatMap(g) == m.flatMap(x => f(x).flatMap(g))).

"Apri la scatola (flatMap), prendi il valore a, applicagli f, e poi rimettilo in una scatola (unit)".

from flatMap and unit

Two implementation strategies in Scala

Aggiungi semplicemente i metodi map e flatMap alla tua classe (magari con le extension methods). Scala ti permetterà subito di usarla nei for.

1. just implement extension methods `flatMap`/`map` for your data-type, and use in for-comprehensions
2. use a type class approach, to get additional operations for free

Definisci un 'given Monad[MioTipo]'. Questo è più potente perché ti permette di scrivere funzioni generiche che accettano qualsiasi Monade (usando i Context Bounds).

The Monad type class

Strategie di Implementazione Numero 2

```scala
object Monads:

  trait Monad[M[_]]:
    def unit[A](a: A): M[A]
    extension [A](m: M[A])
      def flatMap[B](f: A => M[B]): M[B]
      def map[B](f: A => B): M[B] = m.flatMap(a => unit(f(a)))

  object Monad:

    // additional general-purpose operations on monads

    def map2[M[_]: Monad, A, B, C](m: M[A], m2: => M[B])(f: (A, B) => C): M[C] =
      m.flatMap(a => m2.map(b => f(a, b)))

    def seq[M[_]: Monad, A, B](m: M[A], m2: => M[B]): M[B] =
      map2(m, m2)((a, b) => b)

    def seqN[M[_]: Monad, A](stream: Stream[M[A]]): M[A] =
      stream match
        case Cons(h, t) => (h(), t()) match
          case (m, Empty()) => m
          case (m, s) => seq(m, seqN(s))

    // ... many others exist
```

The Optional monad: combing non-empty optionals

```scala
import Monads.*, Monad.*

object Optionals:

  // data structure for optionals
  enum Optional[A]:
    case Just(a: A)
    case Empty()

  // minimal set of algorithms
  object Optional:
    extension [A](m: Optional[A])
      def filter(p: A => Boolean): Optional[A] = m match
        case Just(a) if (p(a)) => m
        case _                 => Empty()

  // extending Optional to be a Monad!
  given Monad[Optional] with
    import Optional.{Just, Empty}

    // unit: just boxing the value
    def unit[A](a: A): Optional[A] = Just(a)

    // flatMap: opens the box if possible, gives the new box
    extension [A](m: Optional[A])
      def flatMap[B](f: A => Optional[B]): Optional[B] =
        m match
          case Just(a) => f(a)
          case Empty() => Empty()
```

Optional monad use

```scala
@main def tryOptionals =
  import Optionals.{*, given} // importing also given terms
  import Optional.*           // importing Optional algorithms

  def optionalRandom(): Optional[Double] =
    Just(java.lang.Math.random()).filter(_ < 0.9)

  // for comprehension: <- just unboxes the Optional if possible
  val m: Optional[Double] = for
    x <- optionalRandom()
    y <- optionalRandom()
    z <- optionalRandom()
  yield x + y + z
  println(m)

  // equivalent formulation by flatMap/map
  val m2: Optional[Double] =
    optionalRandom().flatMap(x =>
      optionalRandom().flatMap(y =>
        optionalRandom().map(z =>
          x + y + z)))
  println(m2)

  // use of some monadic operator
  println:
    map2(Just("rand: "), optionalRandom())(_ + _) // Some("rand: 0.223.."), or Empty()
```

The Sequence monad use

Vediamo il percorso di un singolo valore:
Prende x = 10.
Passa alla riga successiva: prende y = "a".
Passa alla riga successiva: prende z = true.
Arriva allo yield: siccome z è true, restituisce (10, "a").
Torna indietro (grazie alla flatMap): ora prende z = false.
Lo yield restituisce (0, "a").
...e così via per tutte le combinazioni di y e poi di x.

```
1  @main def trySequences =
2    import Sequences.{*, given}, Sequence.*
3
4    // for comprehension: <- just iterates all element,
5    // using each to create a new box
6    val s: Sequence[(Int, String)] = for
7      x <- Cons(10, Cons(20, Nil()))
8      y <- Cons("a", Cons("b", Nil()))
9      z <- Cons(true, Cons(false, Nil()))
10   yield if z then (x, y) else (0, y)
11
12   println(s)
13   // Cons((10,a),Cons((0,a),Cons((10,b),Cons((0,b),
14   // Cons((20,a),Cons((0,a),Cons((20,b),Cons((0,b),Nil()))))))))
```

Nelle slide precedenti abbiamo detto che la Monade definisce il significato del passaggio tra le righe del for. Per una sequenza, il significato è: "Per ogni elemento contenuto in questa scatola, esegui tutto quello che viene dopo."Questo trasforma il for in un sistema di cicli annidati automatici. Non devi scrivere while o foreach; la Monade lo fa per te.

Guarda i generatori:
x ha 2 elementi: (10, 20)
y ha 2 elementi: ("a", "b")
z ha 2 elementi: (true, false)
Il numero totale di combinazioni è 8.

All'interno dello yield, x, y e z sono tipi primitivi (Int, String, Boolean). Puoi usarli in una normale espressione if-then-else. La Monade si occuperà poi di rimettere il risultato dentro una Sequence.

Sequence monad: dealing with combinations of sequences

```scala
object Sequences:

  enum Sequence[A]:
    case Cons(h: A, t: Sequence[A])
    case Nil()

  object Sequence:
    extension [A](s1: Sequence[A])
      def append(s2: Sequence[A]): Sequence[A] = s1 match
        case Cons(h, t) => Cons(h, t.append(s2))
        case Nil()      => s2

  given Monad[Sequence] with
    import Sequence.*
    def unit[A](a: A) = Cons(a, Nil())

    extension [A](m: Sequence[A])
      def flatMap[B](f: A => Sequence[B]): Sequence[B] =
        m match
          case Cons(h, t) => f(h).append(t.flatMap(f))
          case Nil()      => Nil()
```

- f(h): Prende la testa della lista e genera una nuova sequenza.
- .append(...): Attacca a questa nuova sequenza il risultato della flatMap fatta sul resto della lista (t).

IO monad: How do we deal with I/O in pure FP?

```scala
@main def tryMonadIO =

  // Input-output in Scala, though, imperative!
  val s: String = scala.io.StdIn.readLine
  println(s)
  println(s.toInt)

  // Dealing with input/output in pure FP, by monads
  // Operations with side-effects are hidden in the monad
  import IOs.{*, given}, IO.*

  val simpleInteractiveComputation: IO[Int] =
    for
      s <- read()
      _ <- write("Thanks for providing: "+s)
      i <- compute(s.toInt)
      j <- compute(i * 2)
      _ <- write("Result is: "+j)
    yield j

  // desugared version
  val simpleInteractiveComputationByFlatMaps: IO[Int] =
    read()                              .flatMap(s =>
    write("Thanks for providing: "+s) .flatMap(_ =>
    compute(s.toInt)                    .flatMap(i =>
    compute(i * 2)                      .flatMap(j =>
    write("Result is: "+j)              .map(_ =>
      j)))))
```

The IO monad: dealing with interactive computations

```scala
object IOs:

  // data structure for a computation with input/output
  case class IO[A](exec: () => A)

  // minimal set of operations
  object IO:
    def read(): IO[String] = IO(() => scala.io.StdIn.readLine)
    def write[A](a: A): IO[A] = IO(() => { println(a); a })
    def compute[A](a: => A): IO[A] = IO(() => a)
    def nop(): IO[Unit] = compute(())

  // extending IO to be a Monad!
  given Monad[IO] with

    // unit: an IO that just returns the boxed value
    def unit[A](a: A): IO[A] = IO(() => a)

    // flatMap: opens the box, executes, and creates a new box with result
    extension [A](m: IO[A])
      def flatMap[B](f: A => IO[B]): IO[B] =
        m match
          case IO(e) => f(e())
```

The DrawNumber game

```scala
@main def tryDrawGameApp =

  enum Result:
    case Won, Lost

  def drawNumberGame(attempts: Int, draw: Int): IO[Result] =
    if attempts == 0
    then compute(Result.Lost)
    else
      for
        _ <- write("give your number: ")
        d <- read()
        i <- compute(d.toInt)
        res <-
          if (i == draw)
          then
            for
              _ <- write("won")
              r <- compute(Result.Won)
            yield r
          else
            for
              _ <- write(if i > draw then "too high!" else "too low!")
              r <- drawNumberGame(attempts - 1, draw)
            yield r
      yield res

  // starts one run of the game
  drawNumberGame(10, scala.util.Random.nextInt(100))
```

Il "Programmare il punto e virgola" applicato all'IO
In questo codice, il passaggio tra le righe (il ';' invisibile della flatMap) significa:

"Esegui l'effetto precedente (es. stampa), aspetta che finisca, e poi passa il controllo all'effetto successivo."

Senza la monade IO, in un linguaggio "lazy", l'ordine delle stampe e delle letture potrebbe essere completamente casuale! La Monade ripristina l'ordine sequenziale in un mondo funzionale.

Se hai indovinato, viene creato un pacchetto che scrive "won" e restituisce Won.

Se hai sbagliato, viene creato un pacchetto che ti dà un suggerimento ("too high/low") e poi chiama ricorsivamente drawNumberGame per il tentativo successivo.

Qua, usiamo la ricorsione:
La Monade gestisce la catena di chiamate. Ogni tentativo è un nuovo anello della catena di flatMap. Grazie all'ottimizzazione della ricorsione tail (se la monade la supporta), questo non fa crashare la memoria.

The State monad: dealing with evolution of state

```scala
object States:

  // data structure for state (evolution)
  // a state is/has a function evolving S and producing a result A
  case class State[S, A](run: S => (S, A))

  // minimal set of algorithms
  object State:
    // a facility to run the state on an initial ‘s‘
    extension [S, A](m: State[S, A])
      def apply(s: S): (S, A) = m match
        case State(run) => run(s)

  // define a given instance that works on all S, shall use "type lambdas"
  given stateMonad[S]: Monad[[A] =>> State[S, A]] with
    // unit: a state with no evolution, just the result
    def unit[A](a: A): State[S, A] = State(s => (s, a))

    // flatMap: runs the state, use result to create a new state
    extension [A](m: State[S, A])
      override def flatMap[B](f: A => State[S, B]): State[S, B] =
        State(s => m.apply(s) match
          case (s2, a) => f(a).apply(s2)
        )
```

- S: è il tipo dello stato (es. un intero che fa da contatore, o una configurazione).
- A: è il tipo del risultato che l'operazione produce (es. il valore calcolato).
- run: è una funzione che prende lo stato attuale, lo "evolve" e restituisce una coppia: il nuovo stato e il risultato.

Una Monad deve avere un solo parametro di tipo (es. F[A]). Ma State ne ha due: S (lo stato) e A (il risultato). Per farlo funzionare, "fissiamo" S e lasciamo libero A. Quel [A] =>> State [S, A] dice:
- S ha lo stesso tipo in tutta la for-comprehension, ma il suo valore può cambiare riga dopo riga.
- A può cambiare sia come valore che come tipo.

Example: a CounterState ADT, and implementation

```scala
trait CounterState:
  type Counter
  def initialCounter(): Counter
  def inc(): State[Counter, Unit]
  def dec(): State[Counter, Unit]
  def reset(): State[Counter, Unit]
  def get(): State[Counter, Int]
  def nop(): State[Counter, Unit]

object CounterStateImpl extends CounterState:
  opaque type Counter = Int

  def initialCounter(): Counter = 0

  // giving (new_counter, result)
  def inc(): State[Counter, Unit] = State(i => (i + 1, ()));
  def dec(): State[Counter, Unit] = State(i => (i - 1, ()));
  def reset(): State[Counter, Unit] = State(i => (0, ()));
  def get(): State[Counter, Int] = State(i => (i, i));
  def nop(): State[Counter, Unit] = State(i => (i, ()));
```

Using CounterState

```scala
@main def tryCounterState =
  import Monads.*, Monad.*, States.{*, given}, State.*
  val counterState: CounterState = CounterStateImpl
  import counterState.*  // or directly, import CounterStateImpl.*

  println:
    inc().run(initialCounter()) // ((),  1)
  println:
    seq(inc(), inc()).run(initialCounter()) // ((), 2)

  def increment(n: Int): State[Counter, Unit] =
    if n == 0
      then nop()
      else
        for
          _ <- increment(n - 1)
          _ <- inc()
        yield ()

  val session: State[Counter, Int] =
    for
      _ <- inc()
      _ <- reset()
      _ <- increment(5)
      v <- get()
      _ <- reset()
    yield v

  println:
    session.run(initialCounter())  // (5, 0)
```

Ogni riga del for (grazie alla flatMap) prende il contatore aggiornato dalla riga precedente e lo passa alla successiva. Se chiami increment(5), lo stato Counter evolverà 5 volte.

Esegui il get e SALVA il risultato in 'v'

L'uso di '_' (l'underscore) all'interno di una for-comprehension indica che il risultato di quell'operazione non ci serve, quindi lo scartiamo (in questo caso, essendo Unit, non ci serve).

Outline

1. ADTs by alias/opaque types
2. Module types by traits
3. Contextual abstraction by `given`/`using` modifiers
4. Type classes by context bounds
5. `For-yield` and the monad type class
6. A State-monadic little “MVC”

Assuming a SwingFacade and Java integration

```java
import javax.swing.*;
import java.awt.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;

class SwingFunctionalFacade {

    public static interface Frame {
        Frame setSize(int width, int height);
        Frame addButton(String text, String name);
        Frame addLabel(String text, String name);
        Frame showToLabel(String text, String name);
        Frame show();
        Supplier<String> events();
    }

    public static Frame createFrame(){
        return new FrameImpl();
    }

    /*
    private static class FrameImpl implements Frame {
      ...
    }
    */
```

Swing (e Java in generale) nasce come linguaggio imperativo. L'obiettivo qui è creare un Wrapper Funzionale (SwingFunctionalFacade) che tratti la finestra (Frame) come un elemento che "evolve" attraverso uno stato.

WindowState monad (type)

```scala
import Monads.*, Monad.*, States.*, State.*
import u03.extensionmethods.Streams.*

trait WindowState:
  type Window
  def initialWindow: Window
  def setSize(width: Int, height: Int): State[Window, Unit]
  def addButton(text: String, name: String): State[Window, Unit]
  def addLabel(text: String, name: String): State[Window, Unit]
  def toLabel(text: String, name: String): State[Window, Unit]
  def show(): State[Window, Unit]
  def exec(cmd: =>Unit): State[Window, Unit]
  def eventStream(): State[Window, Stream[String]]
```

WindowStateImpl implementation

Invece di modificare la finestra direttamente, ogni metodo restituisce una "ricetta" (un State[Window, Unit]) che dice: "Prendi la finestra attuale, modificala e restituisci la nuova finestra".

```scala
object WindowStateImpl extends WindowState:
  import SwingFunctionalFacade.*

  type Window = Frame


  def initialWindow: Window = createFrame

  def setSize(width: Int, height: Int): State[Window, Unit] =
    State(w => ((w.setSize(width, height)), {}))
  def addButton(text: String, name: String): State[Window, Unit] =
    State(w => ((w.addButton(text, name)), {}))
  def addLabel(text: String, name: String): State[Window, Unit] =
    State(w => ((w.addLabel(text, name)), {}))
  def toLabel(text: String, name: String): State[Window, Unit] =
    State(w => ((w.showToLabel(text, name)), {}))
  def show(): State[Window, Unit] =
    State(w => (w.show, {}))
  def exec(cmd: =>Unit): State[Window, Unit] =
    State(w => (w, cmd))
  def eventStream(): State[Window, Stream[String]] =
    State(w => (w, Stream.generate(() => w.events().get)))
```

MVC

- Input: Prende un'operazione sullo stato del Modello (m1) e una funzione che aggiorna la Vista basandosi sul risultato del Modello (f).
- Output: Restituisce una nuova Monade State che gestisce una coppia di stati (SM, SV).
- Perché è geniale? Permette di dire: "Aggiorna il contatore (Model) ed aggiorna l'etichetta a video (View) come se fossero un'unica operazione atomica".

```
@main def runMVC =
  import Monads.*, Monad.*, States.*, State.*, CounterStateImpl.*, WindowStateImpl.*
  import u03.extensionmethods.Streams.*

  def mv[SM, SV, AM, AV](m1: State[SM,AM], f: AM => State[SV, AV]): State[(SM, SV), AV] =
    State: (sm, sv) =>
      val (sm2, am) = m1.run(sm)
      val (sv2, av) = f(am).run(sv)
      ((sm2, sv2), av)

  def windowCreation(str: String): State[Window, Stream[String]] = for
    _ <- setSize(300, 300)
    _ <- addButton(text = "inc", name = "IncButton")
    _ <- addButton(text = "dec", name = "DecButton")
    _ <- addButton(text = "reset", name = "ResetButton")
    _ <- addButton(text = "quit", name = "QuitButton")
    _ <- addLabel(text = str, name = "Label1")
    _ <- show()
    events <- eventStream()
  yield events

  val controller = for
    events <- mv(seq(reset(), get()), i => windowCreation(i.toString()))
    _ <- seqN(events.map(_ match
        case "IncButton" => mv(seq(inc(), get()), i => toLabel(i.toString, "Label1"))
        case "DecButton" => mv(seq(dec(), get()), i => toLabel(i.toString, "Label1"))
        case "ResetButton" => mv(seq(reset(), get()), i => toLabel(i.toString, "Label1"))
        case "QuitButton" => mv(nop(), _ => exec(sys.exit()))))
  yield ()

  controller.run((initialCounter(), initialWindow))
```

SM: stato Model
SV: stato View
AM: risultato Model
AV: risultato View

Il for del controller viene eseguito una volta sola per costruire una gigantesca catena di flatMap.
- Questa catena è "attaccata" a uno Stream (una lista infinita di click).
- Siccome lo Stream è pigro, la catena di flatMap si mette in attesa.
- Quando arriva un click, la flatMap successiva della catena si "attiva".

crea uno stream che, ogni volta che viene interrogato, chiama w.events().get(): resta lì ferma finché l'utente non clicca un bottone.

Il for del controller non è un ciclo che si ripete. È una definizione di cosa fare

Viene eseguito una sola volta all'apertura della finestra.
Serve a garantire che il contatore parta da 0.

Passiamo lo stato iniziale del contatore e della finestra. Da questo momento, il programma funzionale inizia a prendere gli eventi che arrivano dallo stream, evolvendo i due stati in sincronia.